Aula Extra E3 — NLP + Classificação (Aplicação)

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: Aplicação (notebook); AME §8.1.3 e Ap. A.2–A.3

Objetivos da aula

Ao final desta aula o aluno deve ser capaz de:

- descrever as etapas de **limpeza** e exploração de uma base textual;
- transformar texto em atributos numéricos com ***bag-of-words*** e **TF–IDF**;
- escolher classificadores adequados a dados de texto (Bayes ingênuo, logística penalizada) e justificar a escolha;
- avaliar o classificador com as métricas certas em um problema **desbalanceado** (*spam*);
- montar o ***pipeline*** completo, integrando o que foi visto no curso inteiro.

Em sala

Aula de **síntese**: um problema real (filtrar *spam* de SMS) que costura quase todas as aulas — vetorização de texto, alta dimensão e esparsidade, Bayes ingênuo (Aula 07), métricas em dados desbalanceados (Aula 08) e *pipelines* (Aula 06). Pergunta de abertura: *"um modelo só entende números. Como transformar uma mensagem de texto em algo que ele consiga classificar?"*

1 O problema: filtrar spam

Usaremos a base `spam.csv` (SMS Spam Collection): $\sim$5 574 mensagens de SMS rotuladas como *ham* (legítima, $Y = 0$) ou *spam* ($Y = 1$). É um problema de **classificação binária** (Aula 07) cujas covariáveis são *texto bruto* — e o desafio central é a *representação*: máquinas só processam números.

Atenção

A base é **desbalanceada** (muito mais *ham* que *spam*). Como vimos na Aula 08, a acurácia engana aqui — precisaremos de precisão, revocação e F_1, definidas na seção de avaliação. E os custos são assimétricos: bloquear uma mensagem legítima (falso positivo) costuma ser pior que deixar passar um *spam*.

2 Do texto aos atributos

2.1 Limpeza e normalização

Texto bruto é ruidoso. As etapas usuais de pré-processamento:

- *minusculizar* (`lower`): "Free" e "free" viram o mesmo *token*;
- remover **pontuação** e **números** (conforme o problema);
- *tokenizar*: quebrar a mensagem em palavras (*tokens*);

- remover ***stopwords***: palavras muito frequentes e pouco informativas (“the”, “is”, “and”);
- *stemming/lematização* (opcional): reduzir flexões a um radical (“running” → “run”).

2.2 *Bag-of-words* e a matriz documento–termo

A representação mais simples é o ***bag-of-words*** (saco de palavras): ignora-se a ordem e conta-se a ocorrência de cada palavra. Constrói-se a **matriz documento–termo $\boldsymbol{X}$**, em que a linha i é a mensagem i, a coluna j é um termo do vocabulário, e x_{ij} é a contagem (ou presença binária) do termo j na mensagem i.

Ideia central

Cada mensagem vira um vetor num espaço de dimensão = tamanho do vocabulário — facilmente *milhares* de covariáveis. Mas cada mensagem usa poucas palavras, então $\boldsymbol{X}$ é **esparsa** (quase toda zero). Guarde-a em formato esparso (Apêndice A.3 do [AME]) para economizar memória.

Atenção

Texto é o exemplo canônico de **alta dimensão + esparsidade**: milhares de palavras, mas a maioria irrelevante para a classe. Por isso métodos que *selecionam variáveis* (Lasso, Bayes ingênuo) vão bem, e o KNN vai mal (não seleciona; a distância é dominada por palavras irrelevantes).

2.3 TF–IDF

Contagens cruas dão peso demais a palavras comuns. O **TF–IDF** (*term frequency–inverse document frequency*) reescala cada termo:

$$\text{tf-idf}(t,d) = \underbrace{\text{tf}(t,d)}_{\substack{\text{frequência do termo}\\ \text{no documento } d}} \times \underbrace{\log \frac{N}{\text{df}(t)}}_{\substack{\text{raridade do termo}\\ \text{na coleção}}},$$

onde $\text{df}(t)$ é o número de documentos que contêm t e N o total. Palavras que aparecem em *todo* documento têm IDF baixo (pouco discriminantes); palavras *raras e específicas* ganham peso. Costuma melhorar a classificação de texto.

3 Análise exploratória

Antes de modelar, vale *olhar* os dados: quais as palavras mais frequentes em *ham* vs. *spam*? Em *spam* surgem termos como “free”, “win”, “call”, “txt”, “prize”; em *ham*, vocabulário do dia a dia. Esse contraste já sugere que o *bag-of-words* carrega sinal suficiente para separar as classes.

4 Classificando

Quais métodos usar? Os que prosperam em alta dimensão esparsa:

Bayes ingênuo (multinomial)
: o “cavalo de batalha” do texto. A suposição de independência condicional (Aula 07) é grosseira para palavras, mas funciona surpreendentemente bem e é rápida. A versão *multinomial* modela contagens de palavras. (Lembre da Aula 07: trabalhe em escala logarítmica para evitar *underflow*.)

Logística penalizada (Lasso/Ridge)
discriminativa, robusta, e o Lasso faz *seleção de palavras* — gerando um modelo interpretável (quais palavras pesam para *spam*). É o que o [AME] usa no exemplo das resenhas da Amazon.

Florestas / *boosting*
também aplicáveis; XGBoost costuma ir bem.

Ideia central

Por que Bayes ingênuo combina tão bem com texto? Porque o modelo multinomial sobre contagens de palavras é natural, há pouquíssimos parâmetros por palavra (baixa variância) e a independência condicional, embora falsa, raramente troca a *classe* de maior probabilidade — e, para classificar (Aula 08), basta acertar o lado do corte, não a probabilidade exata.

5 Avaliação

Como o problema é desbalanceado (Aula 08), parta da matriz de confusão, com o *spam* como positivo. A **revocação** é a sensibilidade da Aula 08, VP/P; a **precisão**, que a Aula 08 não usa, é $\mathrm{VP}/(\mathrm{VP}+\mathrm{FP})$, a fração das mensagens bloqueadas que eram *spam*. O F_1 é a média harmônica das duas, $2\,\mathrm{VP}/(2\,\mathrm{VP}+\mathrm{FP}+\mathrm{FN})$, e a **precisão média** (AP) é a média, sobre os *spams*, da precisão no corte que acaba de incluir cada um.

- reporte a **matriz de confusão**, a precisão, a revocação e o F_1 — não só a acurácia;
- para *spam*, a *precisão* sobre a classe "spam" é crítica (não classificar *ham* como *spam*);
- ajuste o **corte** de decisão conforme os custos (Aula 08): se um falso positivo é muito caro, exija probabilidade alta para marcar como *spam*.

Em sala: fechar o curso com uma métrica que discorda das outras duas

A §8 da `Aula prática E3` acrescenta ao saco de palavras o comprimento da mensagem e a proporção de dígitos — duas colunas que separam muito (70,9 contra 138,8 caracteres; 0,4% contra 11,7% de dígitos). Pergunte à turma se o modelo melhora, e depois mostre: acurácia 0,9821 → 0,9862, AUC 0,9879 → 0,9950 e AP 0,9691 → **0,9572**.
A AUC sobe porque as duas colunas arrumam o grosso do ranking; a AP olha a cabeça dele, onde o TF–IDF já ia bem. Como o filtro bloqueia as mensagens de maior escore, quem manda é a AP — e por ela as colunas não pagam o que custam. É um bom encerramento: a métrica tem de ser escolhida *antes*, pela pergunta, ou os mesmos três números justificam três decisões diferentes.

6 O *pipeline* completo (síntese do curso)

Tudo se encaixa num único *pipeline* (Aula 06): vetorização → classificador, com hiperparâmetros escolhidos por validação cruzada (Aula 03) *sem vazamento* — e o vetorizador deve aprender o vocabulário **só no treino**.

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, GridSearchCV
```

```python
from sklearn.metrics import classification_report, confusion_matrix

sms = pd.read_csv("spam.csv", encoding="ISO-8859-1")
X, y = sms["text"], (sms["class"] == "spam").astype(int)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3,
                                          stratify=y, random_state=0)

# vetorizacao + classificador num unico objeto (sem vazamento!)
pipe = Pipeline([
    ("vec", TfidfVectorizer(lowercase=True, stop_words="english")),
    ("clf", MultinomialNB()),
])
grade = {"vec__ngram_range": [(1,1), (1,2)],
         "clf__alpha": [0.1, 0.5, 1.0]} # suavizacao de Laplace
busca = GridSearchCV(pipe, grade, cv=5, scoring="f1").fit(X_tr, y_tr)

y_hat = busca.predict(X_te)
print(confusion_matrix(y_te, y_hat))
print(classification_report(y_te, y_hat)) # precisao, recall, F1
```

O notebook `Aula prática E3` percorre a limpeza, a exploração (palavras mais comuns por classe) e a classificação na base `spam.csv`.

Resumo (e encerramento do curso)

- Texto vira número por ***bag-of-words*** / **TF–IDF** após limpeza (minúsculas, pontuação, *stopwords*).
- A matriz documento–termo é **esparsa e de alta dimensão** — terreno do **Bayes ingênuo** e da **logística penalizada** (Aula 07).
- Avalie com **precisão**, **revocação** e F_1 e ajuste o **corte** pelos custos (Aula 08).
- Tudo num ***pipeline*** com CV sem vazamento (Aulas 03 e 06).

Ideia central: O fio condutor do curso

Este exemplo mostra que os "temas" não são ilhas: representar dados, controlar o balanço viés–variância, estimar o risco honestamente, escolher o método pela estrutura do problema e medir o desempenho com a métrica certa. Foi essa a história do curso inteiro — da regressão linear ao *spam*.

Leitura recomendada. [AME] §8.1.3 (Bayes ingênuo), §3.8 e §8.8 (exemplos com texto: resenhas da Amazon), Apêndice A.2 (manipulação de textos) e A.3 (matrizes esparsas); documentação do `scikit-learn`: *Working With Text Data*.